

Inhoud

Interactie tussen verschillende klassen	1
Associatie Vak - Docent.....	1
Oefening Afdeling – Bediende.....	4
Oefening Punt – Lijnstuk	5
Oefening Digitale Klok - Teller	6

Interactie tussen verschillende klassen.

Laten we eens kijken naar het volgende probleem.

In een school werken een heel aantal docenten. Ook worden er heel wat vakken gegeven. Men wil een overzicht wie welke vakken geeft.

Het probleem is dat men niet op voorhand weet, hoeveel vakken een docent geeft. Dat hangt van de docent in kwestie af.

Er zijn ook andere voorbeelden. Stel dat men wil bijhouden welke artikels een bepaalde klant heeft aangekocht.

Hoe groot dat aantal precies zal worden is onduidelijk en is afhankelijk van de soort klant.

Voor deze soort situaties heeft men in Java collections ontworpen. Een veelgebruikte collection is ArrayList.

Associatie Vak - Docent

Een docent geeft een aantal vakken.

Van een vak willen we weten : de korte omschrijving van het vak, het aantal semesteruren van het vak.

Van een docent willen we weten: de naam, het diploma.

Wanneer we dit voorbeeld op een object georiënteerde manier willen uitwerken, moeten we twee klassen maken.

De eerste klasse is de klasse Docent en de tweede klasse is de klasse Vak.

Er is ook een samenhang/interactie tussen deze twee klassen. Namelijk: een docent geeft verschillende vakken en een vak kan gegeven worden door verschillende docenten.

Als eerste stap vereenvoudigen we het probleem grondig.

Stel dat een docent maar één vak geeft. We moeten dus aan een docent kunnen vragen welk vak hij geeft. Omgekeerd is het niet nodig om aan een vak te vragen welke docent het geeft.

In onderstaand voorbeeld gaan we variabelen definiëren en gebruiken die verwijzen naar objecten van een klasse. Deze variabelen bevatten niet het object zelf, maar zijn pointers of verwijzingen naar het object dat eerder werd aangemaakt. We spreken daarom van referenties. Zolang een variabele van een bepaalde klasse niet wordt ingevuld, bevat deze als waarde de null-pointer.

```
public class Vak {
    private String omschrijving;
    private double aantalUren;

    public Vak(double aantalUren, String omschrijving) {
        this.aantalUren = aantalUren;
        this.omschrijving = omschrijving;
    }

    public String getOmschrijving() {
        return omschrijving;
    }

    public double getAantalUren() {
        return aantalUren;
    }
}
```

```
public class Docent{
    private String naam;
    private String diploma;
    private Vak vak;
    public Docent(String diploma, String naam, Vak vak) {
        this.diploma = diploma;
        this.naam = naam;
        this.vak = vak;
    }
    public void drukAf()
    {
        System.out.println(this.naam + " met diploma "+this.diploma+"
geeft het vak: "+this.vak.getOmschrijving()+" met "+
this.vak.getAantalUren()+" uren");
    }
    public void setVak(Vak vak) {
        this.vak = vak;
    }
}
```

```

public class TestDocent {
    public static void main(String[] args) {
        Vak vak = new Vak(4, "Object georiënteerd programmeren");
        Vak vak2 = new Vak(2, "Programmeren in VB.net");
        Docent docent = new Docent("Master informatica", "Vanderbilt",
vak);
        docent.drukAf();
        docent.setVak(vak2);
        docent.drukAf();
    }
}

```

Output:

Vanderbilt met diploma Master informatica geeft het vak: Object georiënteerd programmeren met 4.0 uren
Vanderbilt met diploma Master informatica geeft het vak: Programmeren in VB.net met 2.0 uren

Uitleg code:

```

public class Docent{
    private String naam;
    private String diploma;
    private Vak vak;

```

- Maak een private variabele aan van het datatype Vak, dit is eigenlijk een referentie of verwijzing of pointer naar een object van klasse Vak. Je kan variabelen declareren van een (zelfgemaakte) klasse, net zoals je variabelen kan declareren van een ander datatype (int, double, ...)
Eigenlijk spreken we hier over een **referentie** of verwijzing naar een bestaand object.

```

public Docent(String diploma, String naam, Vak vak) {
    this.diploma = diploma;
    this.naam = naam;
    this.vak = vak;
}

```

- De constructor Docent heeft als parameter oa vak van klasse Vak. Wanneer je OO werkt, is deze variabele / referentie verplicht van datatype Vak en niet van type String! Je maakt maw de klasse Vak niet vrijblijvend, vanaf het moment dat ze bestaat, gaan we ze ook gebruiken!

```

public void drukAf()
{
    System.out.println(this.naam + " met diploma " + this.diploma +
geeft het vak: " + this.vak.getOmschrijving() + " met " +
this.vak.getAantalUren() + " uren");
}

```

- Vermits vak van het datatype Vak is en dus geen String is, kunnen we de variabele /referentie vak niet zomaar concateneren met de andere strings en primitieve (int, double, ...) variabelen. Door de getOmschrijving()-method vragen we aan het object vak: geef me eens de

omschrijving van dit vak. De omschrijving wordt daardoor geretund en deze is van het datatype String. Met getAantalUren() vragen we aan het object om het aantal uren te geven voor dit vak. Het aantal uren wordt geretund en geconcateneerd aan de string.

Let op! Wanneer we zomaar een object concatenen met strings dan krijgen we de volgende output:

```
Vanderbilt met diploma Master informatica geeft het vak: Vak@e48e1b
Vanderbilt met diploma Master informatica geeft het vak: Vak@12dacd1
```

```
public void setVak(Vak vak) {
    this.vak = vak;
}
```

- We willen de variabele vak een andere inhoud geven. Deze functie (een setter) heeft als parameter een variabele van type Vak.

```
public class TestDocent {
    public static void main(String[] args) {
        Vak vak = new Vak(4, "Object georiënteerd programmeren");
        Vak vak2 = new Vak(2, "Programmeren in VB.net");
```

- Maak eerst twee nieuwe vakken aan.

```
Docent docent = new Docent("Master informatica", "Vanderbilt", vak);
```

- Een object van klasse Docent kan pas worden aangemaakt wanneer er een object van type Vak bestaat. Immers, de constructor van Docent verwacht een object van type Vak.

```
docent.drukAf();
```

```
docent.setVak(vak2);
```

- Wanneer de docent een ander vak geeft, krijgt deze functie een object binnen van type Vak. Ook dit object werd eerder aangemaakt.

```
docent.drukAf();
```

```
}
```

```
}
```

Oefening Afdeling – Bediende

Maak een klasse **Afdeling** aan.

Eigenschappen: naam, locatie

Methods: constructor, getters, setters.

Maak een klasse **Bediende** aan.

Eigenschappen: naam, salaris, afdeling.

!Van welk datatype is de eigenschap afdeling?

Methods:

De constructor geeft nog geen waarde aan de afdeling. Getters en setters.

Een method die het salaris verhoogt met een bepaald bedrag. Test of men het salaris met meer dan 10% wil verhogen. In dat geval wordt het salaris slechts met 10% verhoogd en wordt een foutboodschap getoond.

Een method die alles returnt in de vorm van een String. Deze string ziet er als volgt uit. Wijzig de gegeneerde code van de toString functie volgens wat hier gevraagd wordt.

Mieke verdient 3300 en is werkzaam in Verkoop op de locatie Antwerpen

Maak een Testklasse aan.

Test hierin het volgende.

Maak twee afdelingen aan: boekhouding en verkoop.

Maak twee bedienden aan: mieke en marina.

Zorg dat mieke werkt in de afdeling verkoop en marina in de afdeling boekhouding.

Verhoog het salaris van mieke met een bedrag dat te hoog is.

Verhoog het salaris van marina met een bedrag dat is toegestaan.

Druk vervolgens de eigenschappen af van mieke en marina. Controleer of de salarissen correct verhoogd werden.

Oefening Punt – Lijnstuk

Maak een **klasse Punt**.

Genereer constructor, getters en setters en een toString functie.

Wijzig de de toString functie zodat de volgende String gereturned wordt.

$(x=1.25, y=-5.23)$

Maak een testklasse TestPunt en maak daarin 2 punten aan met respectievelijk de coördinaten (10,10) en (20,30).

Maak een **klasse Lijnstuk** (in dezelfde package als de voorheen gemaakte klasse Punt).

Deze klasse heeft twee attributen, nl. twee variabelen van het type Punt. Het eerste noemen we beginPunt, het tweede eindPunt.

Genereer constructors en getters en setters.

Wijzig de toString-functie zodat je string er als volgt uitziet.

$(10,20) - (20,50)$

Je voegt ook een **methode afstand** toe aan deze klasse. Deze methode retourneert een double, nl. de afstand tussen het eindPunt en het beginPunt. Om die afstand te berekenen, ga je als volgt te werk:

-Voeg eerst aan de **klasse Punt** een methode **afstandTot** toe.

1 parameter : het andere punt.

Je berekent in deze methode dus de afstand van het this-punt tot parameter-punt.

En nu frissen we onze wiskunde op! Stelling van Pythagoras?

Geen nood : hier komt de formule en bovendien is alles te vinden op het internet, dus het enige wat jij nog moet doen is deze formule omzetten naar Java.

Neem de vierkantswortel van $(x2 - x1)^2 + (y2 - y1)^2$.

Hierbij zijn (x2, y2) de coördinaten van het als parameter aangeleverde punt.

(x1, y1) de coördinaten van het this-punt.

Om een vierkantswortel te berekenen in Java gebruik je de methode sqrt uit de klasse Math ->

Math.sqrt(...).

-En hoe gaan we dit nu gebruiken in de methode afstand van klasse Lijnstuk?

Probeer eerst zelf!

Test alles met een testklasse waarin je een lijnstuk aanmaakt op basis van twee Punt-objecten en bereken de afstand(lengte) van het lijnstuk.

Oefening Digitale Klok - Teller

Maak een klasse **DigitaleKlok** en een klasse **Teller**.

De klasse Digitale klok houdt de tijd bij in de vorm H:m:s (uren:minuten:seconden).

Het gebruikt hiervoor drie objecten van het type Teller:

-De teller die de uren voorstelt heeft 0 en 24 als minimum en maximum.

-De andere twee hebben als minimum en maximum 0 en 60.

Wat zijn de private variabelen? Laat nakijken door je docent. Als die goed staan kan je constructoren, getters en setters en de toString functie genereren.

Zorg dat enkel correcte waarden kunnen worden ingegeven voor de tellers.

De toString-functie van de klasse DigitaleKlok retournt de tijd in de volgende vorm : *15:41:19*

Voeg in de klasse DigitaleKlok de methodes **verhoogUur**, **verhoogMinuut** en **verhoogSeconde** toe.

Deze methodes verhogen respectievelijk uur, minuut of seconde met 1, met volgende randvoorwaarden:

Indien seconde 60 zou worden : seconde wordt 0 en minuut wordt met 1 verhoogd.

Indien minuut 60 zou worden : minuut wordt 0 en uur wordt met 1 verhoogd.

Indien uur 24 zou worden : uur wordt 0.

Test alles.